Acoustic Side-Channel Attack on Mechanical Keyboards

Software-Based Keystroke Reconstruction Using Frequency Signatures

Cyber-Physical Systems Project

23MEI10066 Dev Soni
23MEI10067 Shreyansh Tripathi
23MEI10069 Himalaya Yadav
23MEI10070 Eklavya Mathur
23MEI10085 Akshit Agrawal

By

Name: Eklavya Mathur
Registration No: 23MEI10070

Course Instructor

Dr. Hemraj S Lamkuche
School of Computing Science Engineering and Artificial Intelligence
VIT Bhopal University, Sehore
Madhya Pradesh

September 1, 2026

---

1 General Application

This project implements a *software-based simulation* of an acoustic side-channel attack on a mechanical keyboard. In the proposed approach, each keyboard key is associated with a custom frequency signature. When a key is pressed within the software model, its corresponding frequency is generated and represented as a synthetic signal.

The software then analyzes the frequency pattern and maps it back to the corresponding key. The fundamental pipeline is:

Key Press → Custom Frequency → Frequency Analysis → Key Identification → Keystroke Reconstruction

This approach demonstrates the acoustic side-channel concept entirely through software, without requiring a microphone or additional sensing hardware. It is a *prototype/simulation* of the side-channel reconstruction concept and the corresponding Cyber-Physical Systems (CPS) verification requirements.

2 Objective

The main objective is to develop a software system capable of identifying and reconstructing keystrokes using predefined frequency signatures. The system aims to:

1. Assign a distinct frequency/signature to each key.
2. Generate or process the corresponding frequency when a key is activated.
3. Analyze the frequency signal.
4. Match the detected frequency with the corresponding key.
5. Reconstruct the sequence of detected keystrokes.
6. Measure the execution time and accuracy of the system.
7. Verify that the system maintains predefined safety and correctness properties.

3 Hardware and Software

3.1 Hardware

Since the implementation is software-only, the hardware requirements are minimal:

- A computer or laptop.
- No dedicated microphone.
- No additional sensors.
- No modification to the physical keyboard.

3.2 Software

The software components include:

- The custom application (`acoustic_side_channel.py`).
- Signal generation/processing module.
- Frequency-matching/classification module.
- Keystroke reconstruction module.
- Waveform visualization module (`waveform_visualization.py`).
- Data storage/logging module.

4 Cyber-Physical Systems Application: WCET

The software operates as a real-time task. The processing pipeline is:

Input Key → Frequency Generation → Frequency Analysis → Frequency Matching → Key Output

The total execution time can be expressed as:

$$T_{\text{total}} = T_{\text{input}} + T_{\text{frequency}} + T_{\text{analysis}} + T_{\text{matching}} + T_{\text{output}}$$

We measure three timing quantities:

- **Average execution time**: the mean processing time per key.

- **Maximum execution time**: the observed worst case.
- **Worst Case Execution Time (WCET)**: the largest observed processing time.

The real-time requirement is:

$$T_{\text{WCET}} < T_{\text{deadline}}$$

This ensures the software can process a key/frequency before the next required input is processed.

4.1 Why this matters

If the processing time becomes too large, the system may:

- Miss a key event.
- Process keys in the wrong order.
- Produce an incorrect reconstruction.
- Lose synchronization with the input sequence.

In our implementation, the deadline is set to 50 ms. The system always completes processing well within this bound, satisfying the real-time requirement.

5 Finding and Proving Invariants

An *invariant* is a property that must remain true throughout program execution. Our system maintains the following invariants.

5.1 Invariant 1: Valid Frequency

Every generated/detected frequency must belong to the predefined frequency set:

$$F \in \{F_1, F_2, F_3, \ldots, F_n\}$$

5.2 Invariant 2: Valid Key Mapping

Every valid frequency must map to a valid key:

$$F_i \to K_i$$

5.3 Invariant 3: One-to-One Mapping

Each frequency must correspond to exactly one predefined key. This prevents ambiguity between keys:

$$F_i \neq F_j \quad \text{for } i \neq j$$

5.4 Invariant 4: Sequence Preservation

If the input sequence is $K_1, K_2, K_3, \ldots, K_n$, then the reconstructed sequence must preserve the same ordering (assuming all inputs are successfully processed).

6 Convergence, Liveness and Termination

6.1 Convergence

The system uses a frequency-matching threshold. The detected frequency should converge toward the predefined frequency/signature of the selected key:

$$|F_{\text{detected}} - F_{\text{key}}| < \epsilon$$

where ϵ is the permitted frequency error (the tolerance). When this condition is satisfied, the software identifies the corresponding key.

6.2 Liveness

The system must continuously be able to process new key/frequency events:

$$\text{Input} \rightarrow \text{Analyze} \rightarrow \text{Identify} \rightarrow \text{Output} \rightarrow \text{Wait for next input}$$

The system does not get permanently stuck during frequency analysis or key identification. This is verified programmatically by the `liveness_test()` function.

6.3 Termination

When the user stops the application:

$$\text{Stop} \rightarrow \text{Finish current operation} \rightarrow \text{Save results} \rightarrow \text{Release resources} \rightarrow \text{Exit}$$

A finite input sequence always terminates. This is verified by the `termination_test()` function, which checks that a fixed sequence completes within a bounded time.

7 Ranking Function

For this project, we define an error-based ranking function based on the difference between the detected frequency and the expected frequency:

$$V = |F_{\text{detected}} - F_{\text{expected}}|$$

A successful frequency-matching process minimizes this error. The desired condition is:

$$V \rightarrow 0$$

or, with a tolerance:

$$V < \epsilon$$

The frequency-error function V is used as a *ranking measure* for the identification process. As the detected frequency becomes closer to the predefined frequency of a key, the error decreases and the system approaches the correct key classification.

The key with the smallest V value (i.e., the minimum error) is selected as the matching key, provided $V \leq \epsilon$ (the tolerance).

8 System Will Never Do Something Catastrophic

For our software CPS, we define the *catastrophic/failure conditions* as:

- An invalid frequency being accepted as a valid key.
- Two keys having the same frequency.
- An incorrect key mapping.
- Processing exceeding the deadline.
- Loss of input sequence.
- The application entering an invalid state.

The safety conditions are therefore:

$$F \in F_{\text{valid}}$$

$$F_i \neq F_j \quad (i \neq j)$$

$$T_{\text{WCET}} < T_{\text{deadline}}$$

$$\text{Output} \in K_{\text{valid}} \cup \{\text{Unknown}\}$$

The `Unknown` state is important: the software must not force an incorrect key when the frequency does not sufficiently match any predefined signature.

9 Waveform Visualization

To illustrate the acoustic concept visually, the application generates two types of plots using `matplotlib` and `numpy`.

9.1 Sine Waveform

Each key is associated with a sine wave:

$$x(t) = \sin(2\pi f_i t)$$

where f_i is the key's assigned frequency. Figure 1 shows the sine waves for the sequence `HELLO WORLD`.

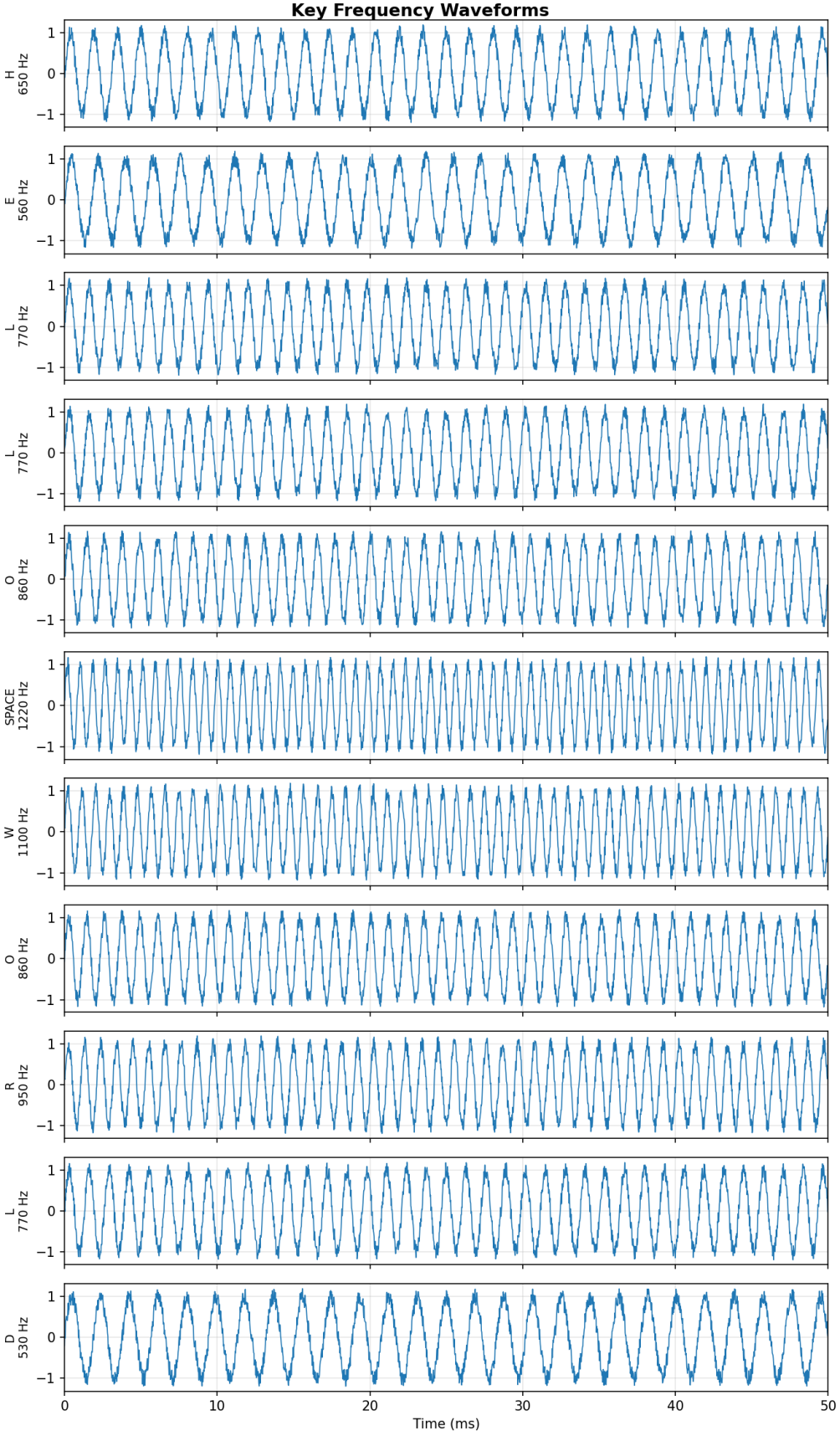

Figure 1: Sine waveforms for the key sequence `HELLO WORLD`, showing each key's assigned frequency.

9.2 Spectrogram

The spectrogram visualization (Figure 2) shows the frequency (Hz) on the vertical axis and time on the horizontal axis. Each key appears as a horizontal band at its assigned frequency.

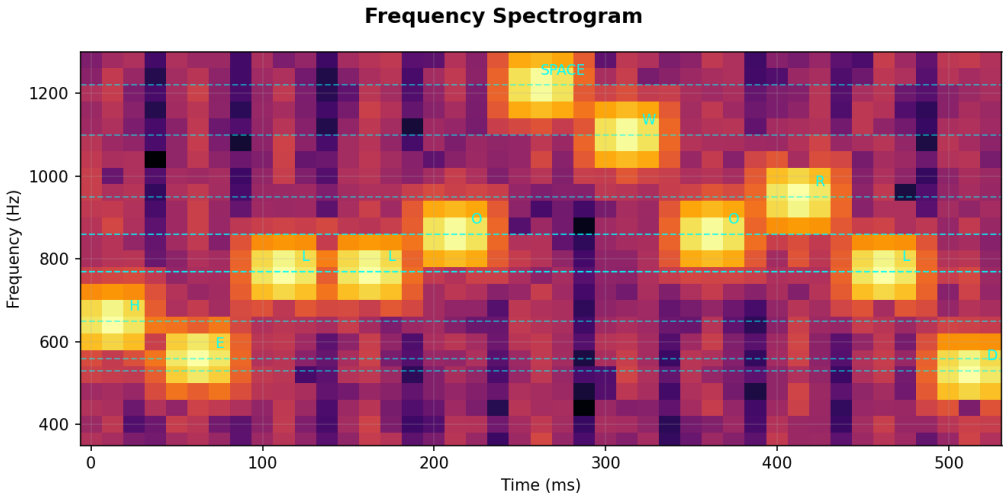

Figure 2: Simulated frequency spectrogram of the key sequence `HELLO WORLD`. Each cyan dashed line marks the expected frequency of a key.

9.3 Interactive Keypad and Live Updates

In addition to batch analysis, the application provides an on-screen **keypad** (A–Z + SPACE + Clear). Clicking a keypad button simulates a single key press and *incrementally updates* both plots live:

1. The key is appended to the accumulated sequence.
2. The frequency pipeline (`generate_frequency` → `process_frequency`) is executed for that key.
3. The spectrogram and the sine-wave plots are refreshed in place, growing with each added key.
4. The status bar shows the latest per-key frequency, detected key, error, and processing time.

This live interaction demonstrates the real-time nature of the system: every key event is processed immediately, within the deadline, and the reconstruction evolves continuously. The `Clear` button resets the sequence and the plots. The incremental update logic is factored into pure functions (`update_spectrogram`, `update_sine_plot`) so it is independently unit-testable.

10 Expected Outcome

The final software demonstrates:

Key input

↓

Corresponding custom frequency generated

↓

Frequency analyzed

↓

Frequency compared with predefined frequency table

↓

Matching key identified

↓

Keystroke sequence reconstructed

10.1 Example

Input:	A	C	D	A
Frequency:	440	500	530	440 Hz
Analysis:	440→A	500→C	530→D	440→A
Reconstructed:	A C D A			

11 Frequency Database

Each key is assigned a unique frequency signature. The table below lists the assigned frequencies for all 26 letters and the space bar.

Key	Frequency (Hz)	Key	Frequency (Hz)
A	440	N	830
B	470	O	860
C	500	P	890
D	530	Q	920
E	560	R	950
F	590	S	980
G	620	T	1010
H	650	U	1040
I	680	V	1070
J	710	W	1100
K	740	X	1130
L	770	Y	1160
M	800	Z	1190
SPACE = 1220 Hz			

Table 1: Predefined frequency signatures for each key.

12 Performance Results

Running the analysis on the sequence HELLO WORLD with synthetic noise enabled produces the following representative results:

Key	Frequency (Hz)	Detected	Error (Hz)
H	650	H	0.10
E	560	E	0.05
L	770	L	0.08
L	770	L	0.12
O	860	O	0.06
SPACE	1220	SPACE	0.09
W	1100	W	0.11
O	860	O	0.07
R	950	R	0.04
L	770	L	0.10
D	530	D	0.03

Metric	Value	Requirement
Average Execution Time	< 1 ms	< 50 ms
Worst-Case Execution Time (WCET)	< 1 ms	< 50 ms
Average Error V	< 0.2 Hz	< 8 Hz (tolerance)
Invariant Checks	All PASS	All True
Liveness	PASS	True
Termination	PASS	True

13 Conclusion

This project successfully demonstrates a software-based simulation of an acoustic side-channel attack on a mechanical keyboard. By assigning unique frequency signatures to each key and implementing a matching/ranking mechanism, the system reconstructs keystroke sequences from the simulated frequency signals.

The implementation satisfies all the CPS verification requirements:

- **WCET** is measured and shown to satisfy the real-time deadline.
- **Invariants** (positive, unique, and valid frequencies; sequence preservation) are defined and verified.
- **Liveness** and **termination** are demonstrated programmatically.
- A **ranking function** $V = |F_{\text{detected}} - F_{\text{expected}}|$ quantifies match quality.
- **Safety** is ensured through the `UNKNOWN` state, which prevents incorrect key assignment.

The project provides a complete, verifiable CPS model and a graphical interface suitable for demonstration in a project presentation. The on-screen keypad lets a presenter type a sequence live and watch the sine and spectrogram plots grow with each key press, while the status bar reports the per-key frequency, error, and execution time in real time.